Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1†
*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan
†Affiliation 2
Email: crg@mail.ntust.edu.tw

***Abstract*—In the Open Radio Access Network (O-RAN) architecture, functional split Option 6 (standardized as nFAPI) provides deployment flexibility over general packet-switched networks by decoupling the MAC and PHY layers, avoiding the very high costs of Time-Sensitive Networking (TSN). However, migrating the MAC layer to a Virtualized Network Function (VNF) in a cloud environment introduces non-deterministic transport and processing latency. Existing open-source implementations rely on static slot-ahead timing configurations, which fail to adapt to unpredictable network jitter, leading to severe scheduling failures and throughput degradation. To address this challenge, this paper proposes an adaptive timing control framework for non-ideal fronthaul environments. The proposed method employs an Exponentially Weighted Moving Average (EWMA) to dynamically estimate network delay and actively adjust the scheduling slot-ahead, ensuring control messages arrive within the strict timing window of the Physical Network Function (PNF). Experimental validations on an OpenAirInterface (OAI) testbed integrated with a commercial O-RAN Radio Unit (O-RU) demonstrate that the proposed adaptive mechanism effectively eliminates synchronization failures, maintains stable Hybrid Automatic Repeat Request (HARQ) round-trip times, and ensures stable system throughput under severe network congestion.**

***Index Terms*—nFAPI, O-RAN, Functional Split, Adaptive Control.**

I. Introduction

A. Background and Motivation

To address the diverse demands of 5G networks, the 3rd Generation Partnership Project (3GPP) and the O-RAN Alliance have standardized the disaggregation of traditional base stations into Open Central Units (O-CUs), Open Distributed Units (O-DUs), and Open Radio Units (O-RUs) [1]. Among the proposed architectures, three functional split configurations have emerged as mainstream: Option 2 (O-CU/O-DU), Option 7.2 (O-DU/O-RU), and Option 6 (O-DU High/Low) [2].

These splits present distinct synchronization requirements and management strategies. Option 2 relies on sequence numbers and deep buffering over general packet-switched networks, making its timing constraints relatively relaxed. Option 7.2 [3], operating within the physical layer, demands strict phase/symbol-level synchronization enforced by hardware Precision Time Protocol (PTP) over Time-Sensitive Networking (TSN). Occupying a unique intermediate position is Option 6 (standardized as nFAPI by the Small Cell Forum (SCF) [4]), which separates the MAC and PHY layers. Option 6 requires frame/slot-level synchronization and focuses on delay management, offering the flexibility to operate over general packet-switched networks without the very high costs of TSN.

The rationale for deploying the MAC layer (O-DU High) in a cloud environment via Option 6 is primarily driven by resource pooling, as Layer 2 processing complexity scales dynamically with user equipment (UE) density [5]. However, migrating the MAC layer to a Virtualized Network Function (VNF) introduces internal processing latency that compresses the available margin for fronthaul transmission delay. Unlike Option 7.2, which rigidly drops out-of-window packets, Option 6 must adaptively manage delays to ensure slot-level synchronization over non-deterministic Ethernet transport. Existing open-source implementations typically employ static slots ahead configurations, which fail to accommodate dynamic network jitter and server processing variations, leading to frequent scheduling failures (e.g., missing slot packet).

This highlights a fundamental gap in current disaggregated RAN designs: the lack of adaptive timing control mechanisms for intermediate-layer splits operating under non-ideal transport environments.

B. Related Works

Recent advancements in Open RAN (O-RAN) architectures have driven significant research into optimizing split-architecture performance and managing network latency. For instance, the X5G testbed proposed by Villa *et al.* [6] demonstrates a state-of-the-art approach by deploying a hardware-accelerated private 5G O-RAN network. By offloading Physical (PHY) layer processing to dedicated NVIDIA GPUs and Mellanox SmartNICs, X5G successfully achieves commercial-grade peak throughputs. However, this high performance relies heavily on expensive, specialized hardware and demands strict, ideal network conditions equipped with hardware-level Precision Time Protocol (PTP) synchronization. Such stringent requirements significantly limit its applicability and deployment flexibility in general packet-switched networks, where unpredictable latency and jitter are prevalent.

Beyond hardware acceleration, software-driven delay management strategies have also been explored from various perspectives. At the intra-node computational level, Lee *et al.* [7] proposed an adaptive Layer 1 transmission strategy that opportunistically advances transmissions when software processing finishes early. From a network orchestration perspective, Adamuz-Hinojosa *et al.* [8] introduced ORANUS,

employing Stochastic Network Calculus (SNC) to provide mathematical latency bounds for ultra-reliable low-latency communications (uRLLC). Moreover, Lv *et al.* [9] emphasized User Equipment (UE) QoS support by integrating MAC resource allocation with high-level network orchestration. While these frameworks [7]–[9] provide robust solutions for processing variance, capacity bounds, and QoS scheduling, they do not directly address the critical synchronization failures caused by unpredictable fronthaul network jitter in intermediate-layer splits.

To address the limitations of high hardware costs and strict synchronization requirements, this paper proposes a cost-effective, software-based solution tailored for non-ideal fronthaul environments. Unlike X5G's reliance on hardware acceleration to bypass latency constraints, our work introduces an adaptive slot-ahead control mechanism based on Exponentially Weighted Moving Average (EWMA) delay management for the nFAPI (Split Option 6) architecture. This approach dynamically tracks and reduces network jitter to actively adapt to unpredictable network congestion using general-purpose Commercial Off-the-Shelf (COTS) servers. Experimental results demonstrate that our proposed method ensures stable Hybrid Automatic Repeat Request (HARQ) Round-Trip Times (RTT) and stable system throughput without the need for high-end specialized equipment or strict hardware synchronization, thereby offering a more flexible and economically viable alternative for O-RAN deployments.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

We identify that the root cause of performance degradation in such systems lies in the inability of the MAC scheduler to adapt its transmission timing to dynamic end-to-end latency variations. Existing open-source implementations, such as OpenAirInterface (OAI), typically employ static slots ahead configurations, which fail to accommodate fluctuating network conditions, leading to frequent synchronization failures and throughput degradation.

To address this issue, we propose an adaptive timing control framework that dynamically adjusts scheduling lead time based on real-time latency estimation. The proposed approach consists of two main components: Node Sync, which aligns frame and slot boundaries between the VNF and PNF, and Delay Management, which applies an Exponentially Weighted Moving Average (EWMA) to smooth the timing information and dynamically adjust the slots ahead to ensure that scheduling messages arrive within the valid timing window.

The main contributions of this paper are summarized as follows:

- **Systematic Analysis of Timing Uncertainty in Split 6:** We characterize the impact of non-deterministic latency, including network jitter and processing delay, on synchronization stability in disaggregated RAN systems.
- **Adaptive Slots Ahead mechanism:** We propose a dynamic timing adjustment mechanism that continuously aligns scheduling decisions with varying transport delays, effectively mitigating timing violations.
- **Experimental Validation on OAI Platform:** We implement the proposed framework in a real-world testbed and demonstrate that it eliminates missing slot packet errors and significantly improves throughput stability under non-ideal fronthaul conditions.
- **Open-Source nFAPI Platform Contribution to Upstream OAI:** We submitted our adaptive timing control and nFAPI modifications as a merge request (MR) to the official OpenAirInterface (OAI) repository. This contribution was merged into the official codebase. It provides a standard open-source nFAPI platform that follows the Small Cell Forum (SCF) specification for future development and testing.

II. System Model

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a stable two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- **Core Network (CN):** The central element responsible for routing, session management, and authenticating user data.
- **O-RAN Central Unit (O-CU):** A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- **O-RAN Distributed Unit (O-DU):** To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- **Switch:** A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.

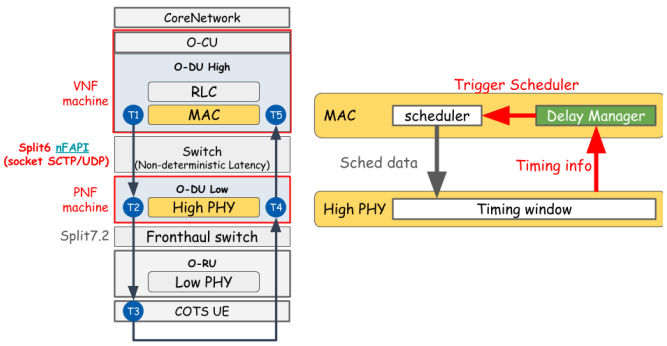


Fig. 1. System architecture and delay management mechanism.

- **O-RAN Radio Unit (O-RU):** A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC and Delay Manager

Within the Media Access Control (MAC) layer of the O-DU High, the core components handling timing mechanisms include the Scheduler and the nFAPI Delay Manager:

- **Scheduler:** Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- **nFAPI Delay Manager:** This resides as the crucial control unit of this delay-aware architecture. Because unpredictable, unstable latency exists in the switch between the VNF and PNF, the Delay Manager continuously receives timing feedback from the lower physical layer. It uses this to dynamically calculate network jitter and actively adjust the **slot-ahead** (S_{ahead}) to trigger the scheduler. This calculation guarantees that the scheduling instructions (`Sched data`) are generated ahead of time to account for network delays.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined, strict time duration. When the scheduling message (specifically the `DL_TTI.request`) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the slot data.
- **Timing Info** (Δt_{arrive})**:** The PNF continuously monitors the exact arrival offset of control messages relative to the transmission deadline. This offset, denoted as Δt_{arrive}, represents the remaining time margin within the timing window. The PNF periodically sends this information back to the Delay Manager in the MAC layer. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following Key Validation Metrics:

- **MAC Layer HARQ RTT** ($T_5 - T_1$)**:** Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- **VNF-PNF DL Latency** ($T_2 - T_1$)**:** Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability:** While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [10], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [11], our system bridges these interfaces.
- **Hybrid System Architecture for E2E Validation:** To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. Proposed Method

This section details the proposed dynamic timing adjustment algorithm. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose an Adaptive Slots Ahead mechanism.

A. Delay Management Architecture

The timing control of this system focuses on the logical aspects of synchronization and delay management, ensuring that the slots ahead on the VNF side can flexibly cope with network jitter without inducing unexpected oscillations. The architecture operates through two primary logical components that interact seamlessly:

1) **Node Sync:** Ensures frame and slot boundary alignment between the VNF and PNF. It monitors the timing offset to maintain synchronization, absorbing transport jitter.
2) **Delay Management:** Focuses on slot-level delay management. It dynamically analyzes the statistical data reported by the PNF through the convergence optimization algorithm to determine the target slot-ahead (S_{next}), guaranteeing that scheduling instructions always meet the PNF's strict timing window constraints.

B. Node Sync

To achieve microsecond-level phase alignment, the system utilizes an IEEE 1588 (PTP) style timestamp exchange. When the VNF receives the *UL_node_sync* message, it calculates the instantaneous timing offset ($offset$) based on four timestamps (t_1 to t_4). To avoid instantaneous spikes and prevent the system from reacting overly sensitively to raw input data, a smoothing preprocessing step is necessary. However, traditional moving averages incur high computational costs to record and maintain historical data within a sliding window. Therefore, the algorithm utilizes an Exponentially Weighted Moving Average (EWMA) model to process the $offset$ and its variation ($error$). This design exponentially decays the weight of historical data while applying only a specific weight to new data, ensuring the system is responsive yet stable. Specifically, it generates an average offset (μ_{offset}) and a mean absolute deviation (dev_{offset}). The smoothing factors α and β are used for these updates. Drawing inspiration from the TCP Congestion Avoidance and Control mechanisms specified in RFC 6298 [12], [13], we set $\alpha = 1/8$ for the mean offset and $\beta = 1/4$ for the jitter variation. This configuration, proven effective in TCP over many years, makes jitter detection more sensitive. Furthermore, to improve execution efficiency, these factors are designed as negative powers of 2 ($\alpha = 2^{-3}$, $\beta = 2^{-2}$), allowing the computation logic to be implemented using highly efficient bitwise shift operations. A dynamic synchronization threshold ($Threshold$) is then derived directly from the estimated jitter (dev_{offset}). If the absolute $offset$ exceeds this $Threshold$, the system breaks the $offset$ down into a slot adjustment (S_{adj}) and a microsecond-level adjustment (μs_{adj}) based on the slot duration (slot_duration). These parameters are subsequently consumed by the VNF Timing Actuator to precisely align the local clock. The process is summarized in Algorithm 1.

Algorithm 1 Node Sync

Require: t_1, t_2, t_3 (Timestamps from PNF *UL_node_sync*), t_4 (VNF local receive timestamp)
Ensure: S_{adj} (Slot adjustment), μs_{adj} (Microsecond adjustment)
Notation:
$offset$ (Instantaneous timing offset), μ_{offset} (EWMA mean offset)
dev_{offset} (EWMA mean absolute deviation / Jitter)
$Threshold$ (Dynamic synchronization threshold)
α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)
slot_duration (Slot duration in µs)
Phase I: Timestamp Processing and Wrap-around Protection
$d_1 \leftarrow t_2 - t_1$
$d_2 \leftarrow t_4 - t_3$
Phase II: Offset and Dynamic Jitter Estimation
$offset \leftarrow (d_1 - d_2)/2$
$error \leftarrow offset - \mu_{offset}$
$\mu_{offset} \leftarrow \mu_{offset} + \alpha \cdot error$
$dev_{offset} \leftarrow dev_{offset} + \beta(|error| - dev_{offset})$
$Threshold \leftarrow dev_{offset}$
Phase III: Phase Alignment
if $|offset| > Threshold$ **then**
    $S_{adj} \leftarrow \lfloor offset/\text{slot\_duration} \rfloor$
    $\mu s_{adj} \leftarrow offset \pmod{\text{slot\_duration}}$
    Apply S_{adj} and μs_{adj} to adjust local clock
else
    State $\leftarrow$ **Synchronized**
    $S_{adj} \leftarrow 0$, $\mu s_{adj} \leftarrow 0$
end if
return $S_{adj}, \mu s_{adj}$

C. Delay Management

To counteract changing network latency and maintain scheduling stability, this paper proposes a dynamic delay management strategy. The algorithm introduces the concepts of predicted risk and historical risk debt to intelligently trade off between throughput and latency. Upon receiving the worst-case late arrival offset (Δt_{arrive}), it first applies the aforementioned EWMA model to calculate the mean delay (μ_{delay}) and jitter deviation (dev_{delay}). Consistent with the Node Sync design inspired by RFC 6298 [12], [13], the smoothing factors are set to $\alpha = 1/8$ and $\beta = 1/4$, allowing for efficient calculation via bitwise shift operations and providing sensitive tracking of network jitter.

The system evaluates the potential tail risk ($Risk_{pred}$) based on the maximum of the observed offset and mean delay, plus the jitter deviation. This risk is continuously accumulated or decayed into a history-weighted risk debt ($Debt_{risk}$). If a deadline violation occurs ($\Delta t_{arrive} > 0$) or high risk is predicted ($Risk_{pred} > 0$), the algorithm immediately increments the current trigger timing (S_{curr}) to preserve throughput. Conversely, if the historical debt is fully cleared ($Debt_{risk} < \epsilon$) and sustained stability is confirmed ($count_{stable} \geq N_{stable}$), the algorithm decrements the advance time to optimize latency. Otherwise, it maintains the current timing and accumulates evidence of stability. The core mechanism is condensed in

Algorithm 2.

Algorithm 2 Delay Management

Require: Δt_{arrive} (Observed timing offset), S_{curr} (Current trigger timing)

Ensure: S_{next} (Trigger timing for the next MAC schedule)

Notation:

μ_{delay} (EWMA mean delay), dev_{delay} (EWMA mean absolute deviation / Jitter)

$Risk_{pred}$ (Predicted timing risk), $Debt_{risk}$ (History-weighted risk debt)

$count_{stable}$ (Consecutive stable arrivals counter)

N_{stable} (Adaptive stability threshold), S_{max} (Maximum advance-time limit)

α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)

ϵ (Debt clearance threshold)

Phase I: Delay, Risk and Debt Estimation

$error \leftarrow \Delta t_{arrive} - \mu_{delay}$

$\mu_{delay} \leftarrow \mu_{delay} + \alpha \cdot error$

$dev_{delay} \leftarrow dev_{delay} + \beta(|error| - dev_{delay})$

$Risk_{pred} \leftarrow \max(\Delta t_{arrive}, \mu_{delay}) + dev_{delay}$ ▷ Evaluate tail risk

$Debt_{risk} \leftarrow Debt_{risk} + \alpha \cdot (\max(0, Risk_{pred}) - Debt_{risk})$ ▷ Accumulate or decay debt

$IsDebtFree \leftarrow (Debt_{risk} < \epsilon)$

Phase II: Trade-off Actuation (Throughput vs. Latency)

if $\Delta t_{arrive} > 0 \vee Risk_{pred} > 0$ **then**

  $S_{next} \leftarrow \min(S_{curr} + 1, S_{max})$ ▷ Preserve throughput

  $count_{stable} \leftarrow 0$

else if $IsDebtFree \wedge (count_{stable} \geq N_{stable})$ **then**

  $S_{next} \leftarrow \max(S_{curr} - 1, 1)$ ▷ Optimize latency

  $count_{stable} \leftarrow 0$

else

  $S_{next} \leftarrow S_{curr}$ ▷ Maintain current timing

  if $Risk_{pred} \leq 0$ **then**

    $count_{stable} \leftarrow count_{stable} + 1$

  end if

end if

return S_{next}

IV. Experimental Results

Experiments were conducted to verify the proposed delay management method in an OpenAirInterface (OAI) nFAPI End-to-End environment. Each reported throughput value represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial). We structure the evaluation into three parts: validating the EWMA mechanism (Section IV-C), benchmarking peak throughput and MAC Round-Trip Time (RTT) against static baselines (Section IV-D), and evaluating stability under network congestion (Section IV-E).

A. Experimental Scenario and rApp Framework

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

TABLE I
Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (`2026.w13`)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

To ensure automated measurement and analysis with consistent data quality, an automated evaluation rApp was developed to operate on the Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [14]. As shown in Fig. 2, this rApp automates the start-up of the VNF and PNF. Once the gNB is established, it automatically connects to the control PC via SSH to toggle the UE's airplane mode using Android Debug Bridge (ADB), and subsequently uses SSH to launch the iPerf server on the Core Network (CN) server. It then executes a series of pre-scheduled iPerf tests automatically. Finally, the rApp collects all log files via O1 SFTP and utilizes Python scripts to plot the following data analysis charts. Crucially, it manages the network latency simulation at $T_5 - T_4$ and $T_2 - T_1$ via Linux Traffic Control (TC) to systematically validate the split-architecture stability.

B. Software Implementation and CI/CD Validation

To show the practical use and compatibility of the proposed adaptive timing control, we have merged our code into the official OpenAirInterface (OAI) repository. In addition to our local automated testbed using the rApp framework, the proposed software implementation was also verified through the official OAI automatic Continuous Integration and Continuous Deployment (CI/CD) pipeline. Specifically, the code passed all image building and cross-compilation tests on x86 and ARM64 systems, including Red Hat Enterprise Linux (RHEL) clusters and NVIDIA Jetson edge platforms. Furthermore, our implementation successfully completed key system-level tests, including 5G Standalone (SA) integration with the FlexRIC controller, real radio frequency (RF) transmissions using USRP B200 and AW2S radio units, F1/N2 handover procedures, and 3GPP-compliant physical layer simulations. These results confirm that our proposed timing control is stable and ready for diverse commercial hardware deployments.

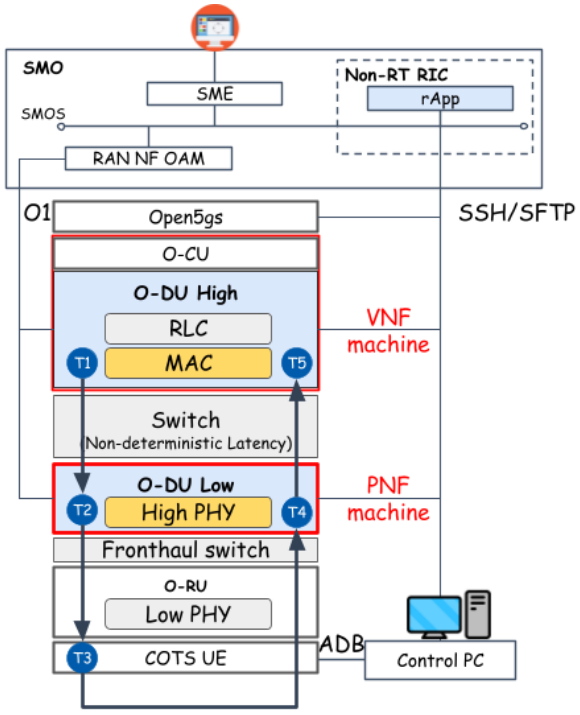
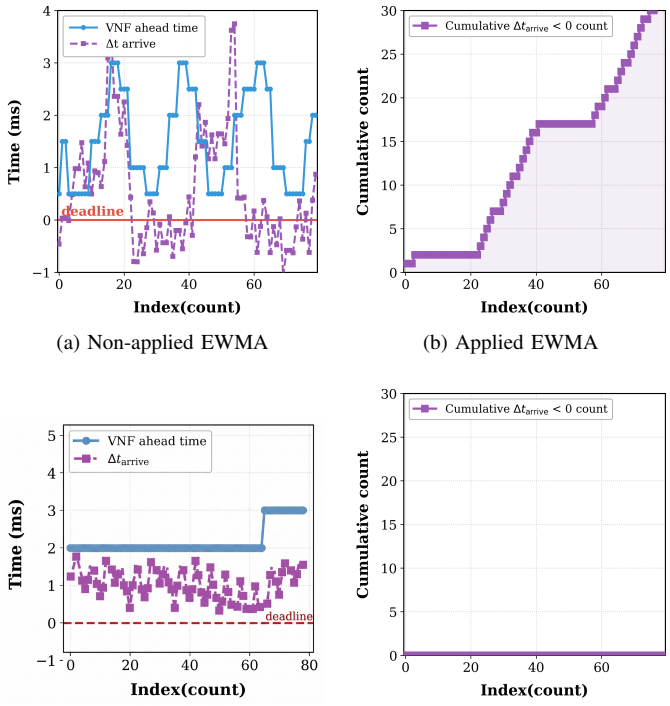


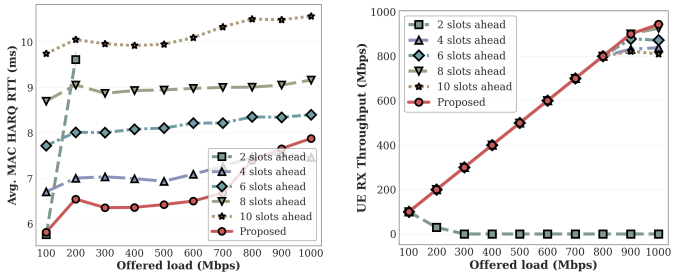
Fig. 2. Architecture of the rApp automated evaluation framework.



(a) Non-applied EWMA (b) Applied EWMA

(c) Trend Detection (d) Proactive Adjustment

Fig. 3. Validation of EWMA processing on PNF Timing Info and proactive adjustment.



(a) Throughput Optimization (b) MAC Latency & Reliability

Fig. 4. Performance evaluation: MAC RTT efficiency and system throughput.

C. Scenario I: Validation of EWMA for PNF Timing Info

This scenario validates the application of an Exponentially Weighted Moving Average (EWMA) to smooth the PNF Timing Info. Without EWMA processing, the raw timing information exhibits high variance due to network jitter, as shown in Fig. 3(a). Applying EWMA effectively filters out this high-frequency jitter (Fig. 3(b)), providing a stable baseline for the dynamic timing adjustment algorithm. Furthermore, as illustrated in Fig. 3(c) and Fig. 3(d), the smoothed data enables the system to reliably detect downward trends in the arrival offset (Δt_{arrive}). Therefore, when Δt_{arrive} begins to exhibit a downward trend, the algorithm can stably judge this condition and actively advance more ahead time to maintain synchronization.

D. Scenario II: Performance Benchmark

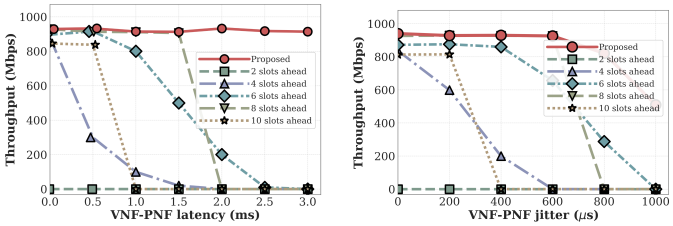
We benchmark the peak throughput and MAC HARQ RTT against a static baseline. Under stable fronthaul conditions, as shown in Fig. 4, the proposed algorithm prioritizes guaranteeing the optimal throughput under different offered loads, consistently maintaining the best throughput performance. Simultaneously, while ensuring this optimal throughput, the algorithm also maximizes the selection of the best slot-ahead to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary overhead.

E. Scenario III: Stability under Network Congestion

To evaluate stability, we inject delay and jitter between the VNF and PNF using Linux TC. As shown in Fig. 5a, static slots ahead configurations suffer from synchronization failures and throughput drops when delay exceeds the timing window, whereas our proposed adaptive mechanism dynamically increases the slots ahead to maintain stable throughput. Additionally, we evaluate the UE DL RX throughput under a saturated 1 Gbps UDP offered load with unpredictable link jitter. As shown in Fig. 5b, while fixed slot ahead configurations experience severe throughput degradation due to late packets, the proposed adaptive method dynamically tracks jitter and secures stable, high throughput at the UE.

V. Conclusion

This paper presented a dynamic timing adjustment framework for disaggregated RAN architectures, specifically focusing on the nFAPI-based Split Option 6 over non-ideal packet-switched networks. Recognizing the limitations of static slots ahead configurations under unpredictable network jitter and



(a) Stability under Delay Congestion

(b) Throughput under Unpredictable Jitter

Fig. 5. System stability and throughput performance under varying network conditions.

server processing delays, we proposed an Adaptive Slots Ahead mechanism based on Exponentially Weighted Moving Average (EWMA) to dynamically track and reduce network latency variations. Experimental validations conducted on the OpenAirInterface (OAI) platform with a commercial Pegatron O-RU demonstrated that the proposed method effectively eliminates synchronization failures and maintains stable system throughput, even under severe network congestion. Unlike existing solutions that rely on expensive hardware-level synchronization or specialized acceleration, our software-based approach offers a cost-effective and flexible alternative for deploying robust O-RAN systems on general-purpose Commercial Off-the-Shelf (COTS) servers.

REFERENCES

[1] *TR 38.801 Study on new radio access technology: Radio access architecture and interfaces*, 3rd Generation Partnership Project (3GPP), 2017, version 14.0.0. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3066

[2] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. D. Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, "A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1637–1678, 2026.

[3] O-RAN Alliance, "O-ran fronthaul working group control, user and synchronization plane specification," O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v11.00, 2023.

[4] Small Cell Forum (SCF), "5g network fapi (nfapi) specification," Small Cell Forum (SCF), Tech. Rep. SCF225, 2020. [Online]. Available: https://www.smallcellforum.org/docs/5g-nfapi-specifications/

[5] X. Foukas *et al.*, "Computational resource consumption of 5g virtualized ran," in *2021 IEEE International Conference on Communications (ICC)*. IEEE, 2021.

[6] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openairinterface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11 305–11 322, 2025.

[7] S.-Q. Lee and M.-S. Lee, "A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb," in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022.

[8] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, "Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 61–70.

[9] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, "Providing ue-level qos support by joint scheduling and orchestration for 5g vran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 51–60.

[10] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.

[11] X. Wang *et al.*, "An open-source implementation of the 5g nr functional split option 6," in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.

[12] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing tcp's retransmission timer," Internet Requests for Comments, RFC Editor, RFC 6298, June 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6298

[13] V. Jacobson, "Congestion avoidance and control," *SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 314–329, aug 1988. [Online]. Available: https://doi.org/10.1145/52325.52356

[14] O-RAN Alliance, "O-ran architecture description," O-RAN Alliance Working Group 1, Tech. Rep. O-RAN.WG1.O-RAN-Architecture-Description, 2024.